

Homework 1 — Gradient Descent

Statistical and Mathematical Methods for AI

Indice

1	Obiettivo dell'homework	2
2	Esercizio 1: GD su una funzione 1D	3
3	Esercizio 2: Backtracking Line Search	7
4	Esercizio 3: GD in 2D	11
5	Esercizio 5: Gradient Descent sulla funzione di Rosenbrock	15

1 Obiettivo dell'homework

Questo homework ha l'obiettivo di costruire una comprensione pratica e quantitativa della **Gradient Descent** (GD), il metodo di ottimizzazione alla base di gran parte del machine learning moderno, esplorandone sistematicamente i comportamenti attraverso cinque esperimenti via via più complessi:

- **Esercizio 1** isola il caso più semplice possibile — una funzione quadratica 1D — per derivare *esattamente* come la scelta dello step size η determini convergenza, oscillazione o divergenza, e per individuare lo step “ottimo” in forma chiusa.
- **Esercizio 2** passa a una funzione **non convessa**, introducendo il **backtracking line search** (condizione di Armijo) come alternativa a uno step fisso, e mostra come punti di partenza diversi possano condurre a minimi locali diversi.
- **Esercizio 3** estende l'analisi a **due dimensioni** con una quadratica mal condizionata, per studiare il fenomeno dello **zig-zag** nelle valli strette e il ruolo del numero di condizionamento $\kappa = \lambda_{\max}/\lambda_{\min}$.
- **Esercizio 4** confronta due strategie di scelta automatica del passo — **exact line search** e **backtracking** — per capire i loro rispettivi compromessi in termini di velocità di convergenza e regolarità dei passi.
- **Esercizio 5** mette alla prova tutto quanto appreso sulla **funzione di Rosenbrock**, un caso classico in cui la curvatura varia drasticamente lungo una valle stretta e curva, rendendo la GD a passo costante particolarmente inefficace.

Il filo conduttore che lega i cinque esercizi è sempre lo stesso: il comportamento della GD — quanto velocemente converge, se oscilla, se diverge — è governato dalla **curvatura** della funzione obiettivo (la sua Hessiana) rispetto allo **step size** scelto, e le strategie di line search (backtracking, exact) servono esattamente ad adattare automaticamente il passo a questa curvatura, che nei casi più interessanti (Esercizi 3, 5) varia da punto a punto o da direzione a direzione.

2 Esercizio 1: GD su una funzione 1D

Consegna

Consider the 1-dimensional function

$$\mathcal{L}(\Theta) = (\Theta - 3)^2 + 1.$$

1. **Compute the Gradient** of $\mathcal{L}(\Theta)$ explicitly.
2. **Implement the Gradient Descent** to optimize $\mathcal{L}(\Theta)$ following what we introduced on the theoretical sections.
3. Test three different constant step sizes:
 - $\eta = 0.05$
 - $\eta = 0.2$
 - $\eta = 1.0$
4. For each choice:
 - Plot the sequence $\Theta^{(k)}$ on the real line (i.e. draw a line and represent, on top of it, the position of all the successive elements $\Theta^{(k)}$).
 - Plot the function values $\mathcal{L}(\Theta^{(k)})$ vs iteration.
 - Comment on convergence, oscillations, and divergence.
5. Relate your observations to the discussion in class about:
 - step-size being too small / too large,
 - the role of convexity,
 - how the “just right” step size leads to fast convergence.

Hint: This function is strictly convex with a unique minimizer at $\Theta^* = 3$.

```
In [1]:  
  
1  def L1(theta):  
2      return (theta - 3.0)**2 + 1.0  
3  
4  def grad_L1(theta):  
5      return 2.0 * (theta - 3.0)  
6  
7  def gradient_descent_1d(grad, theta0, eta, n_iters=30):  
8      theta = theta0  
9      thetas = [theta]  
10     for _ in range(n_iters):  
11         theta = theta - eta * grad(theta)  
12         thetas.append(theta)  
13     return np.array(thetas)  
14  
15 theta0 = -2.0  
16 etas = [0.05, 0.2, 0.5, 1.0, 1.1]  
17 n_iters = 30  
18  
19 trajectories = {eta: gradient_descent_1d(grad_L1, theta0, eta,
```

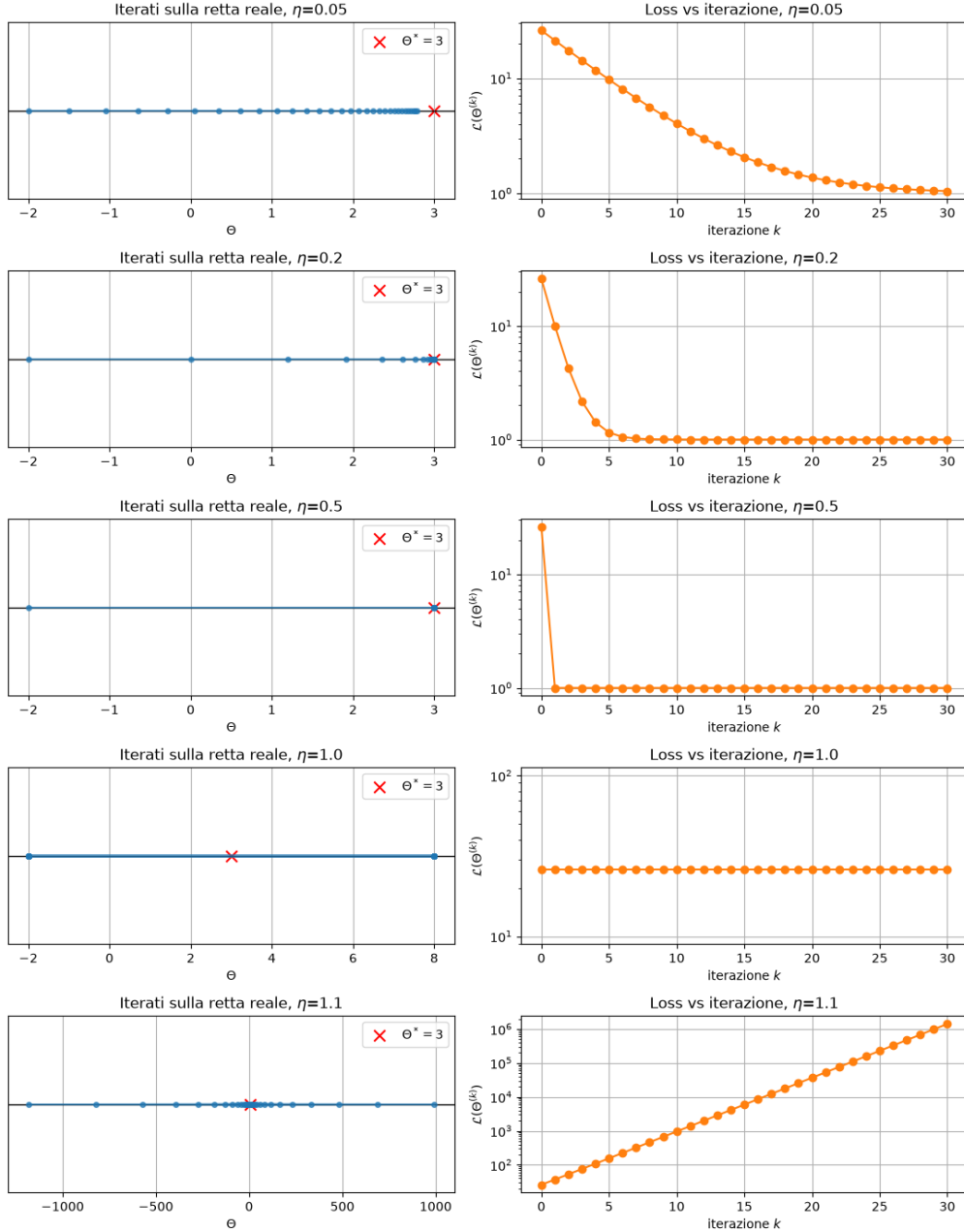
```
n_iters) for eta in etas}
```

L1 e `gradL1` implementano $\mathcal{L}(\Theta) = (\Theta - 3)^2 + 1$ e la sua derivata $\mathcal{L}'(\Theta) = 2(\Theta - 3)$. `gradient_descent_1d` esegue l'aggiornamento $\Theta^{(k+1)} = \Theta^{(k)} - \eta \mathcal{L}'(\Theta^{(k)})$ per `n_iters` passi, salvando l'intera traiettoria (serve per il grafico sulla retta reale, non solo il punto finale). Oltre ai tre η richiesti dalla consegna (0.05, 0.2, 1.0), sono stati aggiunti $\eta = 0.5$ (lo step teoricamente ottimo, si veda sotto) e $\eta = 1.1$ (per osservare la divergenza vera, che con i soli tre valori richiesti non si osserva mai).

In [2]:

```
1  fig, axes = plt.subplots(len(etas), 2, figsize=(11, 2.8*len(etas)))
2
3  for i, eta in enumerate(etas):
4      thetas = trajectories[eta]
5
6      ax = axes[i, 0]
7      ax.axhline(0, color='black', lw=1)
8      ax.plot(thetas, np.zeros_like(thetas), 'o-', markersize=4,
9              alpha=0.7)
9      ax.scatter([3], [0], color='red', marker='x', s=80, label=r'$\Theta^*=3$')
10     ax.set_yticks([])
11     ax.set_title(f'Iterati sulla retta reale, $\eta$={eta}')
12     ax.set_xlabel(r'$\Theta$')
13     ax.legend()
14
15     ax2 = axes[i, 1]
16     ax2.semilogy(range(len(thetas)), L1(thetas), 'o-', color='C1')
17     ax2.set_title(f'Loss vs iterazione, $\eta$={eta}')
18     ax2.set_xlabel('iterazione $k$')
19     ax2.set_ylabel(r'$\mathcal{L}(\Theta^{(k)})$')
20
21 plt.tight_layout()
22 plt.show()
23
24 print(f"{'eta':>6} {'|1-2*eta|':>12} {'theta_30':>14} {'L(theta_30)':>14}")
25 for eta in etas:
26     th = trajectories[eta]
27     print(f"{'eta':>6} {'abs(1-2*eta)':>12.3f} {'th[-1]:>14.6g} {'L1(th[-1]):>14.6g}")
```

Out [2]:



Iterati sulla retta reale (sinistra) e loss vs iterazione in scala logaritmica (destra), per $\eta \in \{0.05, 0.2, 0.5, 1.0, 1.1\}$, partendo da $\Theta^{(0)} = -2$.

	eta	1-2*eta	theta_30	L(theta_30)
1	0.05	0.900	2.78804	1.04493
2	0.2	0.600	3	1
3	0.5	0.000	3	1
4	1.0	1.000	-2	26
5	1.1	1.200	-1183.88	1.40869e+06

Teoria in una riga. Ponendo $e^{(k)} = \Theta^{(k)} - 3$, l'aggiornamento diventa $e^{(k+1)} = (1 - 2\eta) e^{(k)}$, quindi $e^{(k)} = r^k e^{(0)}$ con $r = 1 - 2\eta = 1 - \eta \mathcal{L}''$ (qui $\mathcal{L}'' = 2$). Tutto il comportamento dei cinque pannelli dipende solo da questo numero r , riportato nella tabella sopra come $|1 - 2\eta|$.

Lettura riga per riga.

- $\eta = 0.05 \Rightarrow r = 0.9$: convergenza *monotona* ma lenta. Dopo 30 iterazioni $\Theta^{(30)} = 2.788$, $\mathcal{L} = 1.045$: non ancora arrivato al minimo. Su scala log la curva della loss è una retta, perché il decadimento $e^{(k)} = 0.9^k e^{(0)}$ è geometrico.
- $\eta = 0.2 \Rightarrow r = 0.6$: ancora monotona, ma molto più rapida ($\Theta^{(30)} \approx 3.000$ già dopo poche iterazioni). Non è però lo step ottimo, solo il migliore tra i tre richiesti dalla consegna.
- $\eta = 0.5 \Rightarrow r = 0$: **convergenza esatta in un solo passo** ($\Theta^{(k)} = 3$ per ogni $k \geq 1$), possibile solo perché $\mathcal{L}''$ è costante (funzione esattamente quadratica). È lo step teoricamente ottimo $\eta^* = 1/\mathcal{L}''$.
- $\eta = 1.0 \Rightarrow r = -1$: **caso limite**. $\Theta^{(k)}$ oscilla perpetuamente tra -2 e 8 (segno che si ribalta, modulo costante), senza mai convergere né divergere. Da notare: la loss nel pannello di destra è *perfettamente piatta* a 26 , non oscilla come Θ — perché dipende da $(e^{(k)})^2$, e $(-5)^2 = (+5)^2$.
- $\eta = 1.1 \Rightarrow r = -1.2$: **divergenza oscillante**. Stesso ribaltamento di segno, ma ora $|r| > 1$ amplifica il modulo ad ogni passo: $\Theta^{(30)} \approx -1184$, $\mathcal{L} \approx 1.4 \times 10^6$. Anche qui, su scala log la crescita geometrica appare come una retta (stavolta in salita).

Nota sulla lettura degli assi. Nei pannelli $\eta = 1.0$ e $\eta = 1.1$ l'asse orizzontale del grafico a sinistra si allarga automaticamente (fino a $[-2, 8]$ e $[-1200, 1000]$ rispettivamente): è già un indizio visivo di instabilità prima ancora di leggere i valori numerici.

3 Esercizio 2: Backtracking Line Search

Consegna

Consider the non-convex function

$$\mathcal{L}(\Theta) = \Theta^4 - 3\Theta^2 + 2.$$

1. Implement **Gradient Descent with Backtracking**, using the Armijo condition, considering the `backtracking(...)` function from class.
2. Test different initial points:
 - $\Theta_0 = -2$
 - $\Theta_0 = 0.5$
 - $\Theta_0 = 2$
3. For each starting point, plot:
 - the function curve $\mathcal{L}(\Theta)$ in 1D in the domain $[-3, 3]$,
 - the trajectory of the iterates $\Theta^{(k)}$ overlaid on the curve.
4. Discuss:
 - Why different initializations converge to different minima.
 - How backtracking automatically chooses a suitable step size at each iteration.
 - Situations where constant step size would fail.

In [1]:

```
1 def L2(theta):
2     return theta**4 - 3*theta**2 + 2
3
4 def grad_L2(theta):
5     return 4*theta**3 - 6*theta
6
7 def backtracking(f, grad_f, theta, direction=None, alpha0=1.0, rho
8                 =0.5, c=1e-4):
9     g = grad_f(theta)
10    if direction is None:
11        direction = -g
12    alpha = alpha0
13    f_theta = f(theta)
14    slope = np.dot(np.atleast_1d(g), np.atleast_1d(direction))
15    while f(theta + alpha*direction) > f_theta + c*alpha*slope:
16        alpha *= rho
17        if alpha < 1e-12:
18            break
19    return alpha
20
21 def gd_backtracking(f, grad_f, theta0, n_iters=50, alpha0=1.0, rho
22                   =0.5, c=1e-4):
23     theta = theta0
24     thetas = [theta]
25     alphas = []
26     for _ in range(n_iters):
```

```

25         d = -grad_f(theta)
26         eta = backtracking(f, grad_f, theta, direction=d, alpha0=
                alpha0, rho=rho, c=c)
27         alphas.append(eta)
28         theta = theta + eta*d
29         thetas.append(theta)
30     return np.array(thetas), np.array(alphas)
31
32 theta0_list = [-2.0, 0.5, 2.0]
33 alpha0_list = [1.0, 0.05]
34 n_iters2 = 50
35
36 trajectories2 = {}
37 for a0 in alpha0_list:
38     for t0 in theta0_list:
39         traj, alphas = gd_backtracking(L2, grad_L2, t0, n_iters2,
                alpha0=a0)
40         trajectories2[(a0, t0)] = (traj, alphas)

```

$L2$ e $\text{grad } L2$ implementano $\mathcal{L}(\Theta) = \Theta^4 - 3\Theta^2 + 2$ e $\mathcal{L}'(\Theta) = 4\Theta^3 - 6\Theta$. `backtracking` è la generalizzazione della funzione vista a lezione: con `direction` lasciata al valore di default ($-\nabla f$, come usato ovunque in questo notebook), la condizione di accettazione $f(\theta + \alpha d) > f(\theta) + c\alpha g^T d$ si riduce algebricamente a $f(\theta - \alpha g) > f(\theta) - c\alpha \|g\|^2$, identica a quella della versione di classe. Le uniche aggiunte sono la generalizzazione del parametro `direction` e un controllo di sicurezza (`alpha < 1e-12`) per evitare un ciclo infinito.

A differenza della versione minimale, questa implementazione (tramite `gd_backtracking`) salva anche la sequenza di α scelti ad ogni iterazione (`alphas`) e viene testata con **due** valori di α_0 (1.0 e 0.05), oltre ai tre Θ_0 richiesti — necessario per rispondere in modo quantitativo al punto 4 della consegna (si veda sotto).

In [2]:

```

1  for a0 in alpha0_list:
2      print(f"alpha0 = {a0}")
3      for t0 in theta0_list:
4          traj, alphas = trajectories2[(a0, t0)]
5          print(f"theta0={t0:>5}: primi iterati = {np.round(traj[:5],
                4)}  "
                f"primi passi = {np.round(alphas[:4], 5)}")
6          print(f"{'':13}-> converge a theta={traj[-1]:.4f}, L={L2(
                traj[-1]):.4f}")
7
8      print()

```

Out [2]:

```

1  alpha0 = 1.0
2  theta0= -2.0: primi iterati = [-2.      0.5      1.125    1.2568   1.2068]
3          primi passi = [0.125 0.25   0.125 0.125] -> converge a theta
                =1.2247, L=-0.2500
4  theta0=  0.5: primi iterati = [0.5      1.125    1.2568   1.2068   1.2331]
5          primi passi = [0.25   0.125   0.125 0.125] -> converge a theta
                =1.2247, L=-0.2500
6  theta0=  2.0: primi iterati = [ 2.      -0.5     -1.125   -1.2568  -1.2068]
7          primi passi = [0.125 0.25   0.125 0.125] -> converge a theta
                =-1.2247, L=-0.2500
8

```



```

9  alpha0 = 0.05
10 theta0= -2.0: primi iterati = [-2.      -1.      -1.1     -1.1638 -1.1977]
11      primi passi = [0.05 0.05 0.05 0.05] -> converge a theta
      =-1.2247, L=-0.2500
12 theta0=  0.5: primi iterati = [0.5      0.625   0.7637 0.9037 1.0272]
13      primi passi = [0.05 0.05 0.05 0.05] -> converge a theta
      =1.2247, L=-0.2500
14 theta0=  2.0: primi iterati = [2.       1.       1.1      1.1638 1.1977]
15      primi passi = [0.05 0.05 0.05 0.05] -> converge a theta
      =1.2247, L=-0.2500

```

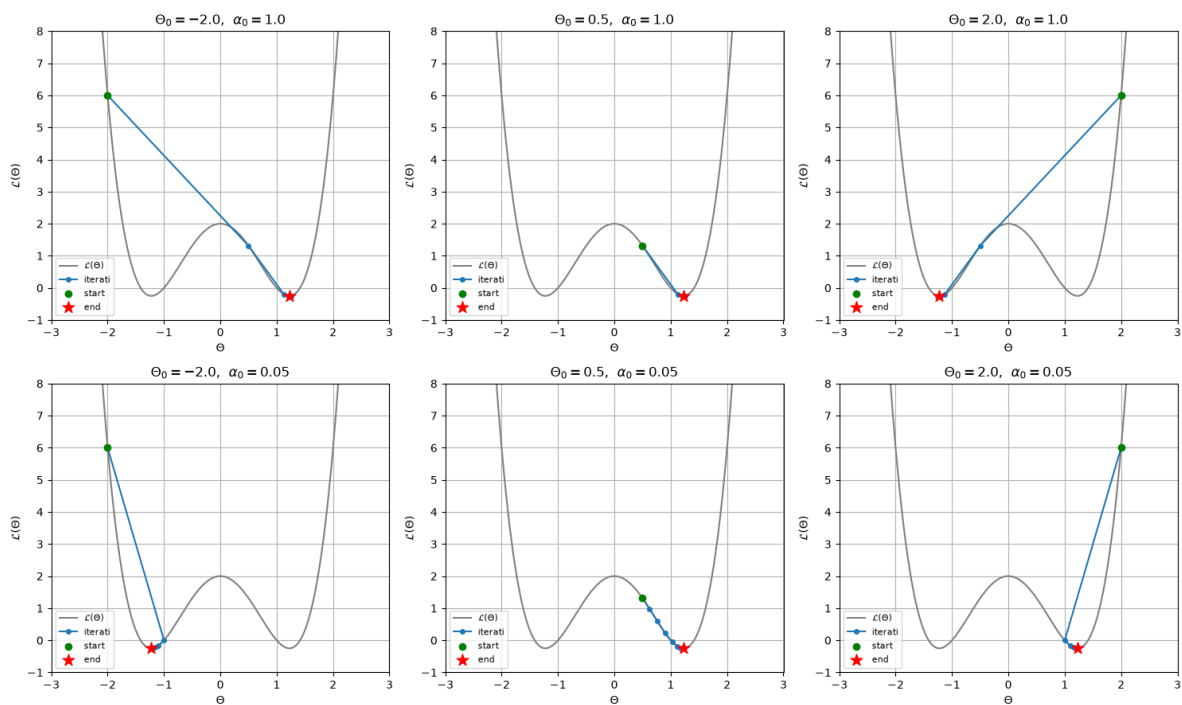
In [3]:

```

1  theta_grid = np.linspace(-3, 3, 400)
2  fig, axes = plt.subplots(len(alpha0_list), 3, figsize=(15, 4.5*len(
    alpha0_list)))
3  for i, a0 in enumerate(alpha0_list):
4      for j, t0 in enumerate(theta0_list):
5          ax = axes[i, j]
6          traj, _ = trajectories2[(a0, t0)]
7          ax.plot(theta_grid, L2(theta_grid), color='gray', label=r'$
            \mathcal{L}(\Theta)$')
8          ax.plot(traj, L2(traj), 'o-', color='C0', markersize=4,
            label='iterati')
9          ax.scatter([traj[0]], [L2(traj[0])], color='green', zorder
            =5, label='start')
10         ax.scatter([traj[-1]], [L2(traj[-1])], color='red', marker=
            '*', s=120, zorder=5, label='end')
11         ax.set_title(rf'$\Theta_0={t0}$,  $\alpha_0={a0}$')
12         ax.set_xlabel(r'$\Theta$'); ax.set_ylabel(r'$\mathcal{L}(\Theta)$')
13         ax.set_xlim(-3, 3); ax.set_ylim(-1, 8)
14         ax.legend(fontsize=8)
15     plt.tight_layout()
16     plt.show()

```

Out [3]:



Traiettorie per i tre Θ_0 richiesti e due valori di α_0 (riga sopra: $\alpha_0 = 1.0$; riga sotto: $\alpha_0 = 0.05$).

La funzione. $\mathcal{L}'(\Theta) = 4\Theta^3 - 6\Theta = 2\Theta(2\Theta^2 - 3)$: punti stazionari in $\Theta = 0$ (massimo locale, $\mathcal{L}''(0) = -6 < 0$) e $\Theta = \pm\sqrt{3/2} \approx \pm 1.2247$ (minimi globali, $\mathcal{L} = -0.25$).

Un fenomeno non ovvio. Con $\alpha_0 = 1.0$, i punti $\Theta_0 = -2$ e $\Theta_0 = 2$ convergono al minimo *opposto* rispetto a quello che ci si aspetterebbe (rispettivamente $+1.2247$ e -1.2247): guardando “primi passi”, il primo α accettato da $\Theta_0 = -2$ è 0.125 , che produce uno spostamento di 2.5 , sufficiente a scavalcare interamente la barriera in $\Theta = 0$ e atterrare nel bacino opposto. Con $\alpha_0 = 0.05$ invece i “primi passi” restano *sempre* 0.05 (il ciclo `while` non deve mai dimezzare): i passi restano piccoli, non scavalcano mai la barriera, e ciascun Θ_0 converge al minimo dalla propria parte.

Perché inizializzazioni diverse convergono a minimi diversi. La GD è un metodo *locale*: segue il gradiente nel punto corrente senza informazione globale sulla funzione. Cade nel bacino di attrazione determinato dalla traiettoria seguita — che dipende sia da Θ_0 sia, come appena mostrato, da quanto è aggressivo il passo di prova iniziale α_0 . La regola “ogni punto converge al minimo più vicino” vale quindi solo se α_0 è abbastanza piccolo da non scavalcare le barriere vicine.

Come il backtracking sceglie il passo automaticamente. Ad ogni iterazione parte da α_0 e lo dimezza finché non vale la condizione di Armijo $f(\theta + \alpha d) \leq f(\theta) + c\alpha \nabla f(\theta)^T d$. Questo adatta il passo alla curvatura locale — ma, come appena visto, “adattarsi alla curvatura locale” garantisce solo che la loss scenda, non che il passo resti prudente rispetto a barriere vicine.

Quando uno step costante fallirebbe. $\mathcal{L}''(\Theta) = 12\Theta^2 - 6$ varia molto lungo il dominio: negativa vicino a $\Theta = 0$, positiva e crescente altrove. Uno step costante abbastanza piccolo da essere stabile vicino ai minimi sarebbe troppo lento nella zona piatta centrale; uno abbastanza grande da muoversi rapidamente lì rischierebbe di divergere vicino ai minimi. Il backtracking risolve questo compromesso rivalutando la curvatura locale ad ogni passo.

4 Esercizio 3: GD in 2D

Consegna

Consider the quadratic function:

$$\mathcal{L}(\Theta) = \frac{1}{2} \Theta^T A \Theta, \quad A = \begin{bmatrix} 1 & 0 \\ 0 & 25 \end{bmatrix}.$$

1. Implement Gradient Descent in 2D, as seen in class.
2. Plot the **level sets** of L (using the helper code provided in the lecture notes) and overlay the iterates $\Theta^{(k)}$ for:

- $\eta = 0.02$,
- $\eta = 0.05$ (borderline),
- $\eta = 0.1$ (diverging).

To overlay the iterates on the plot, simply add a `plt.plot(...)` function taking as input the coordinates $\Theta_1^{(k)}$ and $\Theta_2^{(k)}$ for each k .

3. Comment on:

- the elongated ellipses produced by ill-conditioning,
- the gradient direction compared to the level set lines,
- zig-zag behaviour for large condition numbers,
- the relation to the lecture discussion on slow convergence in narrow valleys.

In [1]:

```
1  A3 = np.array([[1.0, 0.0], [0.0, 25.0]])
2
3  def L3(theta):
4      return 0.5 * theta @ A3 @ theta
5
6  def grad_L3(theta):
7      return A3 @ theta
8
9  def gd_2d(grad, theta0, eta, n_iters=50):
10     theta = np.array(theta0, dtype=float)
11     thetas = [theta.copy()]
12     for _ in range(n_iters):
13         theta = theta - eta * grad(theta)
14         thetas.append(theta.copy())
15     return np.array(thetas)
16
17 theta0_3 = np.array([4.0, 1.0])
18 etas3 = [0.02, 0.05, 0.1]
19 n_iters3 = 40
20
21 trajectories3 = {eta: gd_2d(grad_L3, theta0_3, eta, n_iters3) for
22                  eta in etas3}
23
24 print(f"{'eta':>6} {'1-eta*1':>10} {'1-eta*25':>10} {'||theta_40
25       ||':>14}")
26 for eta in etas3:
```

```

25     print(f"{eta:>6} {1-eta:>10.3f} {1-25*eta:>10.3f} "
26           f"{np.linalg.norm(trajectories3[eta][-1]):>14.4g}")

```

Out[1]:

	eta	1-eta*1	1-eta*25	theta_40
1	0.02	0.980	0.500	1.783
2	0.05	0.950	-0.250	0.514
3	0.1	0.900	-1.500	1.106e+07

L3 e `grad_L3` implementano $\mathcal{L}(\Theta) = \frac{1}{2}\Theta^T A \Theta$ e $\nabla \mathcal{L}(\Theta) = A\Theta$, con $A = \text{diag}(1, 25)$. Poiché A è già diagonale, il problema si disaccoppia automaticamente in due direzioni indipendenti (gli autovettori di A coincidono con gli assi Θ_1, Θ_2): ciascuna evolve come $\Theta_i^{(k)} = (1 - \eta\lambda_i)^k \Theta_i^{(0)}$, con $\lambda_1 = 1$ (direzione piatta) e $\lambda_2 = 25$ (direzione ripida, $\kappa = 25$). I valori $1 - \eta\lambda_i$ stampati sopra sono esattamente i fattori di contrazione di ciascuna componente.

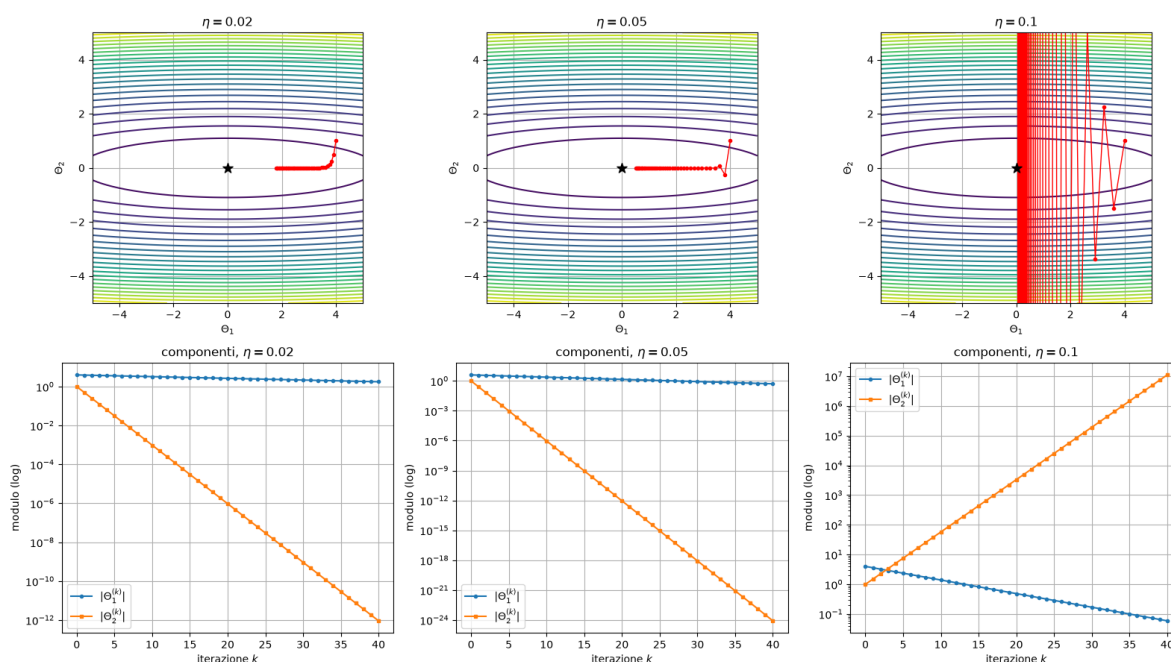
In [2]:

```

1  x1 = np.linspace(-5, 5, 300)
2  x2 = np.linspace(-5, 5, 300)
3  X1, X2 = np.meshgrid(x1, x2)
4  Z3 = 0.5*(A3[0,0]*X1**2 + A3[1,1]*X2**2)
5
6  fig, axes = plt.subplots(2, 3, figsize=(16, 9))
7  for j, eta in enumerate(etas3):
8      traj = trajectories3[eta]
9
10     ax = axes[0, j]
11     ax.contour(X1, X2, Z3, levels=25, cmap='viridis')
12     ax.plot(traj[:,0], traj[:,1], 'o-', color='red', markersize=3,
13             linewidth=1)
14     ax.scatter([0],[0], color='black', marker='*', s=100, zorder=5)
15     ax.set_title(rf'$\eta$={eta}$')
16     ax.set_xlabel(r'$\Theta_1$'); ax.set_ylabel(r'$\Theta_2$')
17     ax.set_xlim(-5,5); ax.set_ylim(-5,5)
18     ax.set_aspect('equal')
19
20     ax2 = axes[1, j]
21     ax2.semilogy(np.abs(traj[:,0]) + 1e-300, 'o-', markersize=3,
22                  label=r'$|\Theta_1^{(k)}|$')
23     ax2.semilogy(np.abs(traj[:,1]) + 1e-300, 's-', markersize=3,
24                  label=r'$|\Theta_2^{(k)}|$')
25     ax2.set_xlabel('iterazione $k$'); ax2.set_ylabel('modulo (log)')
26     ax2.set_title(rf'componenti, $\eta$={eta}$')
27     ax2.legend()
28
29  plt.tight_layout()
30  plt.show()
31
32  for eta in etas3:
33      print(f"eta={eta}: segni di Theta_2 nelle prime 8 iterate = "
34            f"{np.sign(trajectories3[eta][:8,1]).astype(int)}")

```

Out [2] :



Sopra: curve di livello con traiettoria sovrapposta, per $\eta = 0.02, 0.05, 0.1$. Sotto: modulo di $\Theta_1^{(k)}$ e $\Theta_2^{(k)}$ in scala logaritmica, stesse tre colonne.

- 1 `eta=0.02: segni di Theta_2 nelle prime 8 iterate = [ 1 1 1 1 1 1 1 1]`
- 2 `eta=0.05: segni di Theta_2 nelle prime 8 iterate = [ 1 -1 1 -1 1 -1 1 -1]`
- 3 `eta=0.1: segni di Theta_2 nelle prime 8 iterate = [ 1 -1 1 -1 1 -1 1 -1]`

Ellissi allungate per il mal condizionamento. Le curve di livello di $\frac{1}{2}\Theta^T A \Theta$ con $A = \text{diag}(\lambda_1, \lambda_2)$ sono ellissi con semiassi $\propto 1/\sqrt{\lambda_i}$: qui il rapporto tra i semiassi è $\sqrt{25}/\sqrt{1} = 5$, esattamente $\sqrt{\kappa}$. Si vede bene nel pannello sopra: le ellissi sono molto schiacciate lungo Θ_2 .

Gradiente vs linee di livello. Il gradiente $\nabla \mathcal{L} = A\Theta = (\Theta_1, 25\Theta_2)$ è sempre perpendicolare alle curve di livello (fatto esatto, vale per qualsiasi funzione differenziabile), ma non punta verso il centro quando le ellissi sono allungate: la componente lungo Θ_2 viene amplificata di un fattore 25 rispetto a quella lungo Θ_1 , causando l'overshoot lungo la direzione stretta.

Zig-zag per κ grande. Dalla tabella: soglia di stabilità sulla coordinata Θ_2 pari a $\eta < 2/25 = 0.08$.

- $\eta = 0.02$: entrambi i fattori positivi (0.98 e 0.5) $\Rightarrow$ nessuna alternanza di segno (confermato: segni sempre +1), traiettoria che si appiattisce dritta sull'asse Θ_1 .
- $\eta = 0.05$: fattore su Θ_2 negativo (-0.25) $\Rightarrow$ segni alternati, ma $|-0.25| < 1$ quindi lo zig-zag si smorza rapidamente (si veda il crollo di $|\Theta_2^{(k)}|$ da 10^0 a 10^{-21} in 40 iterazioni).
- $\eta = 0.1$: fattore -1.5 , sopra soglia $\Rightarrow$ segni alternati e $|-1.5| > 1$: divergenza oscillante, $\|\Theta^{(40)}\| \approx 1.1 \times 10^7$.

Nota: segno alternato da solo non basta a distinguere convergenza da divergenza — è il *modulo* del fattore a deciderlo, non il segno.

Relazione con la lentezza nelle valli strette. La soglia di stabilità è dettata dalla direzione più ripida ($\lambda_2 = 25$), ma con η scelto per quella soglia la direzione piatta ($\lambda_1 = 1$) ha fattore $1 - \eta \approx 0.98$, vicinissimo a 1: progresso lentissimo. È visibile nei pannelli $\eta = 0.02$ e 0.05 : la curva blu ($|\Theta_1^{(k)}|$) resta quasi piatta per tutte le 40 iterazioni, mentre quella arancione ($|\Theta_2^{(k)}|$) crolla rapidamente — esattamente il conflitto tra stabilità (dettata da $\lambda_{\max}$) e velocità (dettata da $\lambda_{\min}$) discusso a lezione.

5 Esercizio 5: Gradient Descent sulla funzione di Rosenbrock

Consegna

The 2-dimensional Rosenbrock function is defined as:

$$\mathcal{L}(\Theta) = (1 - \Theta_1)^2 + 100(\Theta_2 - \Theta_1^2)^2,$$

with a unique global minimum at $\Theta^* = (1, 1)$, $\mathcal{L}(\Theta^*) = 0$.

Despite having only one minimum, its landscape is **extremely challenging**:

- The valley is long and curved.
 - The condition number varies drastically across the region.
 - Gradients can point almost orthogonally to the valley direction.
 - Gradient Descent may zig-zag and make very slow progress toward the solution.
1. Implement the function and its gradient to run the Gradient Descent algorithm. Test its performance using both **constant step size** and the **backtracking algorithm**.
 2. Plot the level sets of $\mathcal{L}$, with a wide range ($\Theta_1 \in [-2, 2]$, $\Theta_2 \in [-1, 3]$) and enough contour lines to visualize the narrow valley. Choose at least four initial points: $(-1.5, 2)$, $(-1, 0)$, $(0, 2)$, $(1.5, 1.5)$. Run both GD a passo costante ($\eta = 10^{-3}, 10^{-4}, 10^{-5}$) e con backtracking.
 3. Per ogni inizializzazione e metodo: plot della traiettoria sulla mappa dei livelli, plot della loss vs iterazione, e discussione se il metodo entra nella valle, quanto ci mette a girare nella direzione giusta, se fa zig-zag, se il passo diventa troppo piccolo (costante) o troppo grande (divergenza).

```
In [1]:  
  
1  def rosenbrock(theta):  
2      t1, t2 = theta  
3      return (1 - t1)**2 + 100*(t2 - t1**2)**2  
4  
5  def grad_rosenbrock(theta):  
6      t1, t2 = theta  
7      dt1 = -2*(1 - t1) - 400*t1*(t2 - t1**2)  
8      dt2 = 200*(t2 - t1**2)  
9      return np.array([dt1, dt2])  
10  
11 def gd_constant_2d(f, grad, theta0, eta, n_iters):  
12     theta = np.array(theta0, dtype=float)  
13     thetas, losses, nevals = [theta.copy()], [f(theta)], [0]  
14     n = 0  
15     with np.errstate(over='ignore', invalid='ignore'):  
16         for _ in range(n_iters):  
17             theta = theta - eta * grad(theta)  
18             n += 1  
19             if not np.all(np.isfinite(theta)):  
20                 print(f"    -> divergenza (overflow) alla  
21                     iterazione {n}")  
22                 break  
23             thetas.append(theta.copy())
```

```

23         losses.append(f(theta))
24         nevals.append(n)
25     return np.array(thetas), np.array(losses), np.array(nevals)
26
27 def gd_backtracking_nd(f, grad, theta0, n_iters, alpha0=1.0, rho
=0.5, c=1e-4):
28     counter = {'n': 0}
29     def f_counted(x):
30         counter['n'] += 1
31         return f(x)
32     theta = np.array(theta0, dtype=float)
33     thetas, losses, nevals, alphas = [theta.copy()], [f(theta)],
[0], []
34     for _ in range(n_iters):
35         d = -grad(theta)
36         eta = backtracking(f_counted, grad, theta, direction=d,
alpha0=alpha0, rho=rho, c=c)
37         alphas.append(eta)
38         theta = theta + eta*d
39         thetas.append(theta.copy())
40         losses.append(f(theta))
41         nevals.append(counter['n'])
42     return np.array(thetas), np.array(losses), np.array(nevals), np
.array(alphas)

```

`rosenbrock`, `grad_rosenbrock`: implementano $\mathcal{L}$ e $\nabla\mathcal{L}$ come dati dalla consegna. A differenza degli esercizi precedenti, qui la Hessiana dipende dal punto Θ (non è costante come nel caso quadratico): il condizionamento varia lungo tutta la valle.

`gd_constant_2d` include un controllo esplicito di **divergenza** (`np.isfinite`) che interrompe il ciclo e stampa un messaggio se Θ esplode a overflow — necessario perché con $\eta = 10^{-2}$ (aggiunto oltre ai tre richiesti) la sequenza diverge in pochissime iterazioni. Entrambe le funzioni contano anche il numero di valutazioni di $\mathcal{L}/\nabla\mathcal{L}$ (`nevals`), per un confronto a parità di costo computazionale — essenziale perché il backtracking richiede più valutazioni per iterazione dello step costante.

In [2]:

```

1  r1 = np.linspace(-2, 2, 400)
2  r2 = np.linspace(-1, 3, 400)
3  R1, R2 = np.meshgrid(r1, r2)
4  Zr = (1 - R1)**2 + 100*(R2 - R1**2)**2
5
6  def plot_rosenbrock_levels(ax):
7      ax.contour(R1, R2, Zr, levels=np.logspace(-0.5, 3.5, 25), cmap=
'viridis', norm='log')
8      ax.plot(r1, r1**2, ':', color='gray', lw=1, label=r'fondo valle
$\Theta_2=\Theta_1^2$')
9      ax.scatter([1],[1], color='red', marker='*', s=120, zorder=5,
label=r'$\Theta^*=(1,1)$')
10     ax.set_xlabel(r'$\Theta_1$'); ax.set_ylabel(r'$\Theta_2$')
11     ax.set_xlim(-2,2); ax.set_ylim(-1,3)
12
13 fig, ax = plt.subplots(figsize=(6,5))
14 plot_rosenbrock_levels(ax)
15 ax.set_title('Curve di livello della funzione di Rosenbrock')

```

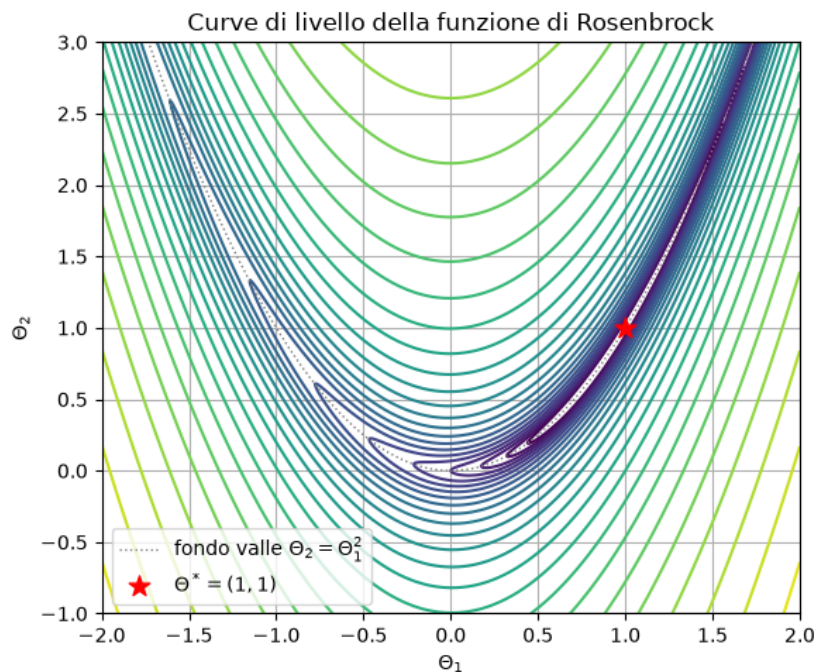


```

16 ax.legend()
17 plt.tight_layout()
18 plt.show()

```

Out [2]:



Curve di livello (scala log) della funzione di Rosenbrock, con il fondo valle $\Theta_2 = \Theta_1^2$ evidenziato.

La valle stretta e curva è ben visibile: le curve di livello si "comprimono" attorno alla parabola $\Theta_2 = \Theta_1^2$ (linea tratteggiata), che indica dove $100(\Theta_2 - \Theta_1^2)^2 = 0$, cioè dove si annulla il termine dominante della funzione. Il minimo $\Theta^* = (1, 1)$ giace esattamente su questa curva.

In [3]:

```

1  starts = {
2      'A (-1.5, 2)': np.array([-1.5, 2.0]),
3      'B (-1, 0)':   np.array([-1.0, 0.0]),
4      'C (0, 2)':    np.array([0.0, 2.0]),
5      'D (1.5, 1.5)': np.array([1.5, 1.5]),
6  }
7
8  const_etas = [1e-2, 1e-3, 1e-4, 1e-5]
9  n_iters_const = 3000
10 n_iters_bt = 200
11
12 results = {}
13 for name, theta0 in starts.items():
14     print(f"[{name}]")
15     results[name] = {'const': {}, 'bt': None}
16     for eta in const_etas:
17         results[name]['const'][eta] = gd_constant_2d(

```

```

18         rosenbrock, grad_rosenbrock, theta0, eta, n_iters_const
19     )
20     results[name]['bt'] = gd_backtracking_nd(
21         rosenbrock, grad_rosenbrock, theta0, n_iters_bt)
22
23     print()
24     print(f"{'start':15s} {'metodo':20s} {'iter':>6s} {'val. di L':>10s}
25           {'theta finale':>22s} {'L finale':>12s}")
26     for name in starts:
27         for eta in const_etas:
28             traj, loss, nev = results[name]['const'][eta]
29             print(f"{name:15s} {'const eta='+format(eta,'g'):20s} {len(
30                 loss)-1:>6d} "
31                 f"{nev[-1]:>10d} {np.array2string(traj[-1], precision
32                 =3):>22s} {loss[-1]:>12.4e}")
33             traj, loss, nev, alphas = results[name]['bt']
34             print(f"{name:15s} {'backtracking':20s} {len(loss)-1:>6d} {nev
35                 [-1]:>10d} "
36                 f"{np.array2string(traj[-1], precision=3):>22s} {loss
37                 [-1]:>12.4e}")
38             print(f"{'':15s} (passi: min={alphas.min():.1e}, max={alphas.
39                 max():.1e})")

```

Out[3]:

```

1  [A (-1.5, 2)]
2      -> divergenza (overflow) alla iterazione 9
3  [B (-1, 0)]
4      -> divergenza (overflow) alla iterazione 7
5  [C (0, 2)]
6      -> divergenza (overflow) alla iterazione 9
7  [D (1.5, 1.5)]
8      -> divergenza (overflow) alla iterazione 7
9
10 start          metodo          iter  val. di L          theta finale      L
11   finale
12 A (-1.5, 2)    const eta=0.01          8          8 [1.747e+255  1.151e+170]
13   inf
14 A (-1.5, 2)    const eta=0.001        3000        3000          [0.78  0.607]
15   4.8669e-02
16 A (-1.5, 2)    const eta=0.0001        3000        3000          [-1.249  1.567]
17   5.0626e+00
18 A (-1.5, 2)    const eta=1e-05         3000        3000          [-1.405  1.982]
19   5.7896e+00
20 A (-1.5, 2)    backtracking           200        2412          [1.715  2.945]
21   5.1248e-01
22   (passi: min=4.9e-04, max=6.2e-02)
23 B (-1, 0)      const eta=0.01          6          6 [-2.304e+180  1.385e+120]
24   inf
25 B (-1, 0)      const eta=0.001        3000        3000          [0.877  0.769]
26   1.5087e-02
27 B (-1, 0)      const eta=0.0001        3000        3000          [9.967e-03  9.967e-05]
28   9.8017e-01
29 B (-1, 0)      const eta=1e-05         3000        3000          [-0.502  0.259]
30   2.2607e+00
31 B (-1, 0)      backtracking           200        2083          [0.845  0.713]
32   2.4157e-02
33   (passi: min=2.0e-03, max=7.8e-03)
34 C (0, 2)       const eta=0.01          8          8 [1.359e+111  9.738e+073]
35   inf
36 C (0, 2)       const eta=0.001        3000        3000          [0.895  0.8 ]
37   1.1091e-02
38 C (0, 2)       const eta=0.0001        3000        3000          [0.447  0.197]
39   3.0684e-01

```

```

26 C (0, 2) const eta=1e-05 3000 3000 [0.207 0.043]
    6.2835e-01
27 C (0, 2) backtracking 200 1920 [0.721 0.521]
    7.7652e-02
    (passi: min=2.0e-03, max=6.2e-02)
28 D (1.5, 1.5) const eta=0.01 6 6 [3.277e+173 3.773e+115]
    inf
29 D (1.5, 1.5) const eta=0.001 3000 3000 [1.096 1.202]
    9.2688e-03
30 D (1.5, 1.5) const eta=0.0001 3000 3000 [1.239 1.535]
    5.7037e-02
31 D (1.5, 1.5) const eta=1e-05 3000 3000 [1.257 1.582]
    6.6326e-02
32 D (1.5, 1.5) backtracking 200 2272 [1.224 1.499]
    5.0338e-02
33 (passi: min=9.8e-04, max=3.9e-03)
34

```

In [4]:

```

1 fig, axes = plt.subplots(2, 2, figsize=(13, 11))
2 colors_const = {1e-2:'tab:purple', 1e-3:'tab:red', 1e-4:'tab:orange',
3                 '1e-5':'tab:brown'}
4 for ax, (name, theta0) in zip(axes.flat, starts.items()):
5     plot_rosenbrock_levels(ax)
6     for eta in const_etas:
7         traj, loss, nev = results[name]['const'][eta]
8         ax.plot(traj[:,0], traj[:,1], '-', color=colors_const[eta],
9                 lw=1.2, alpha=0.8,
10                 label=fr'const $\eta$={eta:g}')
11     traj_bt_, loss_bt_, nev_bt_, alphas_bt_ = results[name]['bt']
12     ax.plot(traj_bt_[:,0], traj_bt_[:,1], '-', color='blue', lw
13             =1.6, label='backtracking')
14     ax.scatter([theta0[0]], [theta0[1]], color='green', s=60, zorder
15               =6, label='start')
16     ax.set_title(f'Start {name}')
17     ax.legend(fontsize=8)
18
19 plt.tight_layout()
20 plt.show()
21
22 fig, axes = plt.subplots(4, 2, figsize=(13, 18))
23 for row, (name, theta0) in enumerate(starts.items()):
24     for col, xlabel in enumerate(['iterazione $k$', 'valutazioni di
25                                  $\mathcal{L}/\nabla\mathcal{L}$']):
26         ax = axes[row, col]
27         for eta in const_etas:
28             traj, loss, nev = results[name]['const'][eta]
29             x = np.arange(len(loss)) if col == 0 else nev
30             ax.semilogy(x, loss, color=colors_const[eta], lw=1.2,
31                         label=fr'const $\eta$={eta:g}')
32             traj_bt_, loss_bt_, nev_bt_, alphas_bt_ = results[name]['bt']
33             x = np.arange(len(loss_bt_)) if col == 0 else nev_bt_
34             ax.semilogy(x, loss_bt_, color='blue', lw=1.6, label='
35                         backtracking')
36             ax.set_title(f'Start {name}')
37             ax.set_xlabel(xlabel)
38             ax.set_ylabel(r'$\mathcal{L}(\Theta^{(k)})$ (log)')
39             ax.legend(fontsize=8)

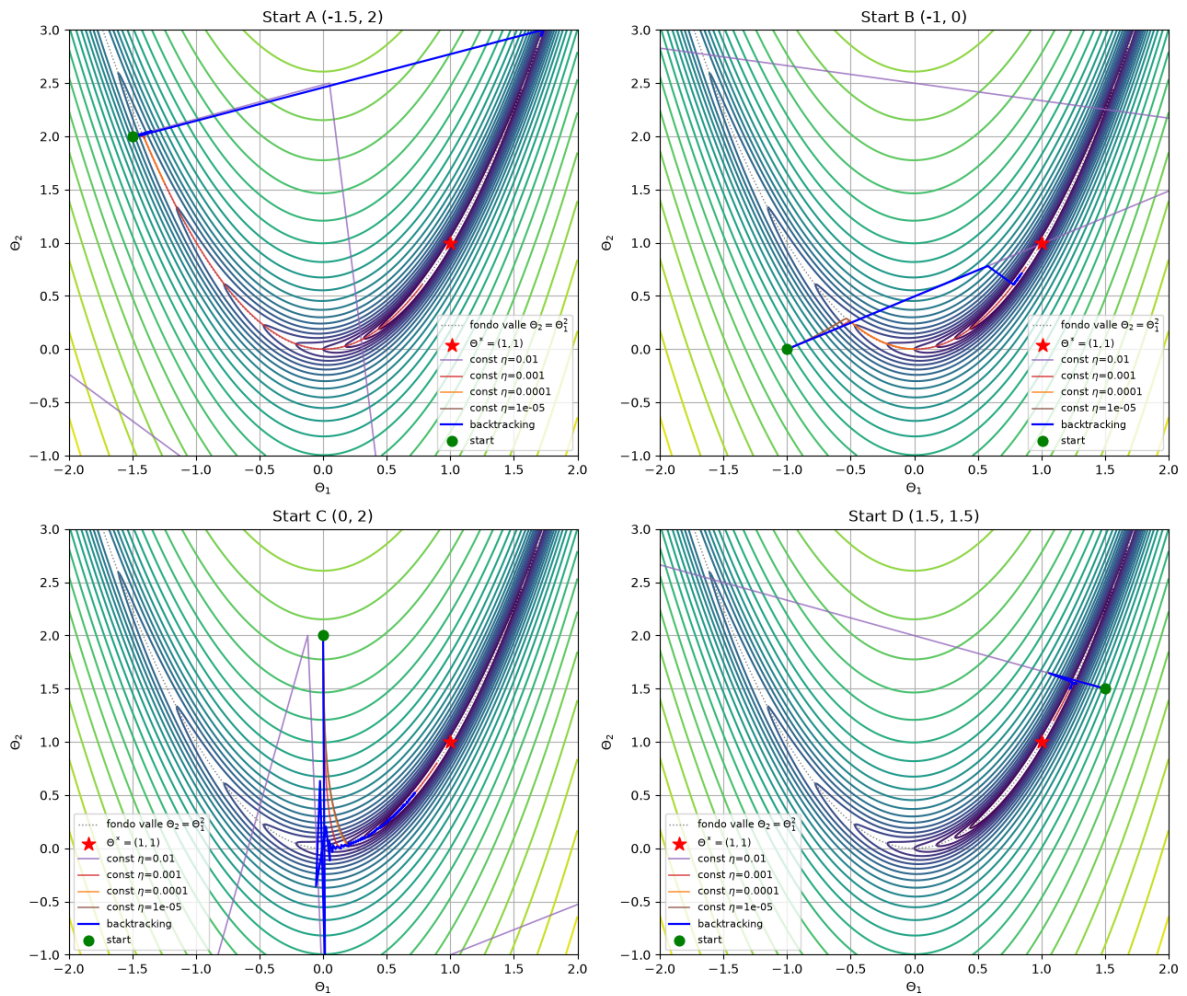
```

```

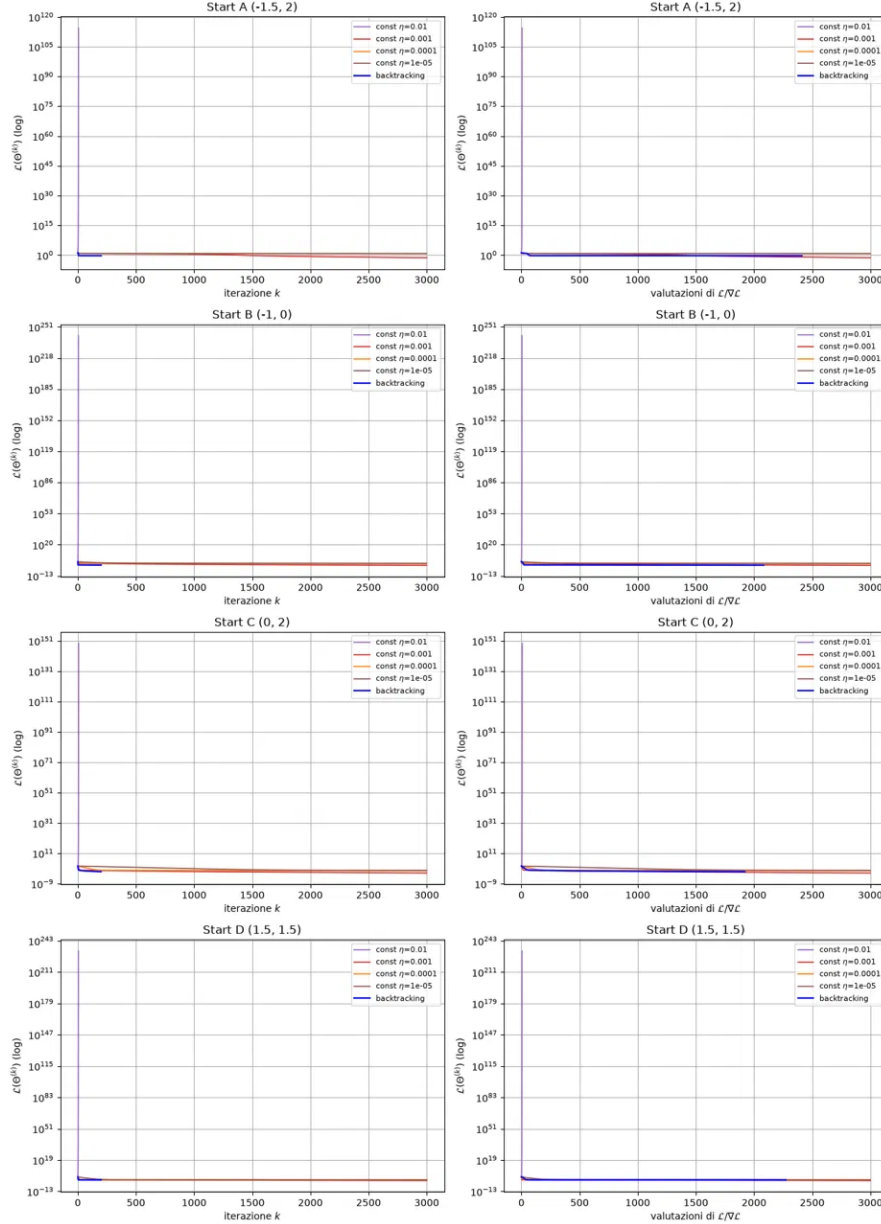
34 plt.tight_layout()
35 plt.show()

```

Out [4]:



Traiettorie sulla mappa dei livelli, per i quattro punti iniziali richiesti ($\eta = 10^{-2}$ aggiunto per osservare la divergenza).



Loss vs iterazione (sinistra) e vs numero di valutazioni di $L/\nabla L$ (destra) --- quest'ultimo è il confronto equo tra step costante e backtracking.

Se il metodo entra nella valle. Sì, per tutti i metodi non divergenti: nella Figura sopra le curve rosse ($\eta = 0.001$) e arancioni ($\eta = 0.0001$) seguono da vicino il fondo valle tratteggiato in tutti e quattro i pannelli. Verifica numerica diretta: per A con $\eta = 10^{-4}$, $\Theta_{\text{finale}} = (-1.249, 1.567)$, e $(-1.249)^2 = 1.560 \approx 1.567$ — il punto è già sul fondo valle, anche se ancora lontano dal minimo.

Quanto ci mette a “girare” nella direzione giusta. Il caso più istruttivo è il backtracking su C (0, 2): nella traiettoria blu si vede una fitta serie di rimbalzi verticali quasi immobili vicino a $\Theta_1 \approx 0$ prima di imboccare la valle — coerente col fatto che C ha la soglia locale di stabilità più permissiva tra i quattro punti (curvatura trasversale relativamente più debole), il che rende più difficile per il backtracking individuare subito l’orientamento della valle.

Se il metodo fa zig-zag. Sì, più visibile nel backtracking che nello step costante: coi

passi costanti piccoli lo zig-zag c'è (è strutturale, il gradiente dentro una valle curva non è mai perfettamente allineato con la direzione di avanzamento) ma è troppo fine per essere visibile a questa risoluzione. Il backtracking, con passi che variano di due ordini di grandezza nello stesso run (per A: da 4.9×10^{-4} a 6.2×10^{-2}), rende lo zig-zag grossolano e visibile, specialmente dove la curvatura cambia bruscamente.

Passi troppo piccoli o troppo grandi.

- $\eta = 0.01$: sempre troppo grande, diverge in tutti e quattro i punti entro 6–9 iterazioni.
- $\eta = 0.0001$ e 10^{-5} : troppo piccoli — con lo stesso budget di 3000 iterazioni non arrivano lontanamente vicino a $\eta = 0.001$: per A, la loss finale peggiora da 4.9×10^{-2} a 5.06 a 5.79 scegliendo passi via via più cauti. Non è instabilità, è l'opposto: così stabile da avanzare troppo poco.
- $\eta = 0.001$: il punto di equilibrio tra i quattro valori testati.
- Il backtracking aggira il problema: guardando la loss vs valutazioni (pannello di destra), a parità di costo computazionale è competitivo o migliore dello step costante ottimale, senza dover indovinare un η fisso valido per l'intera traiettoria.